Lecture 4: Deep Learning Fundamentals

Overview

- Artificial neural network
- Fundamental building block: tensors
- Represent tabular data with tensors

Artificial Neural Network

From Biological to Artificial Neurons

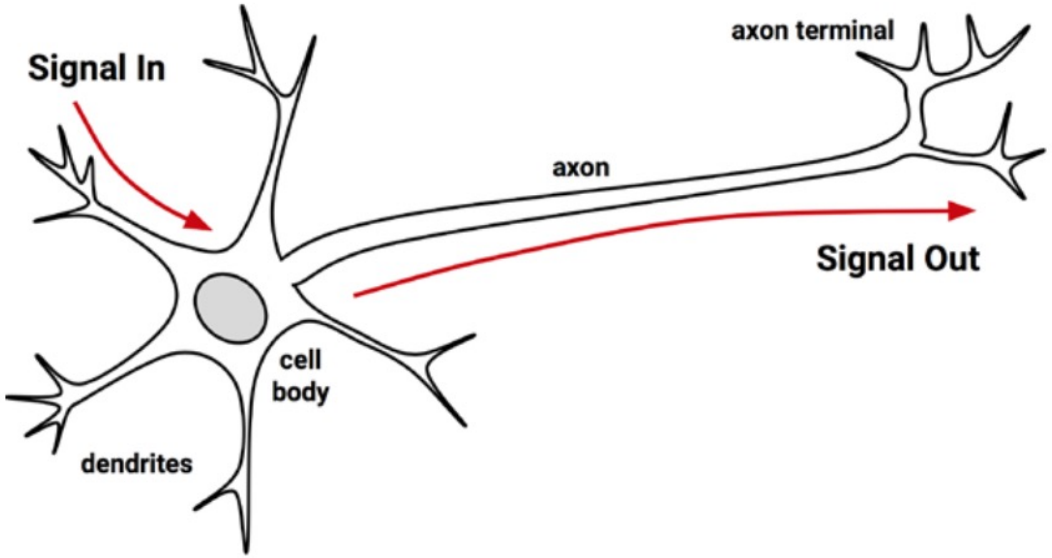
- Neural networks, are computational models inspired by biological neural networks, particularly, they are inspired by the behavior of neurons and the electrical signals they convey between input, processing, and output from the brain.

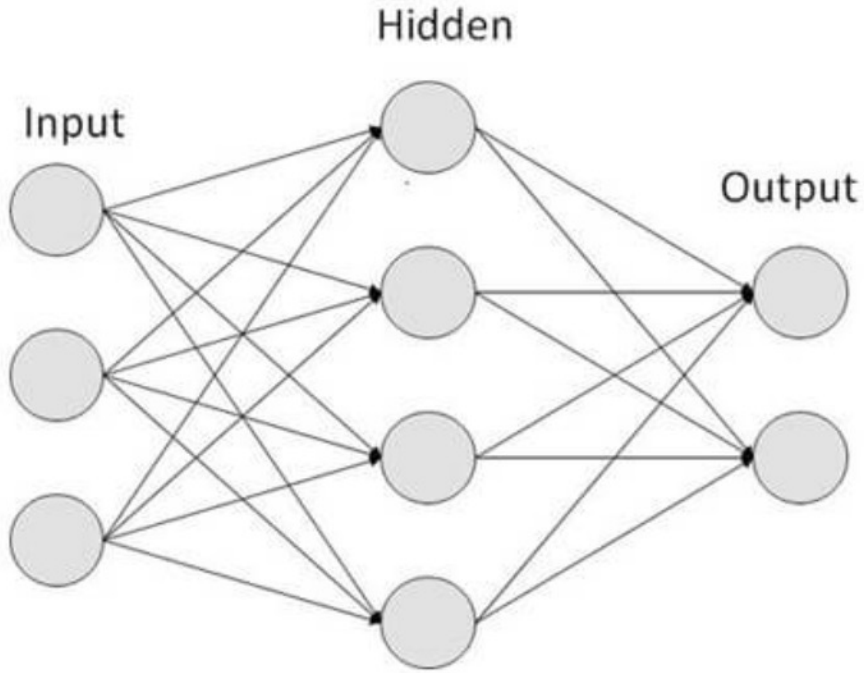
From Biological to Artificial Neurons

- An Artificial Neural Network (ANN) models the relationship between a set of input signals and an output signal
  - Derived from our understanding of how a biological brain responds to stimuli from sensory inputs
    - A brain uses a network of interconnected cells called **neurons** to create a massive parallel processor
  - ANN uses a network of **artificial neurons** or **nodes** to solve learning problems
- ANNs are versatile learners that can be applied to nearly any learning tasks
  - Classification, numeric prediction, and even unsupervised pattern recognition

From Biological to Artificial Neurons

- Nature has inspired millions of innovations-the human brain has inspired the creation of neural networks.





Artificial Neural Network

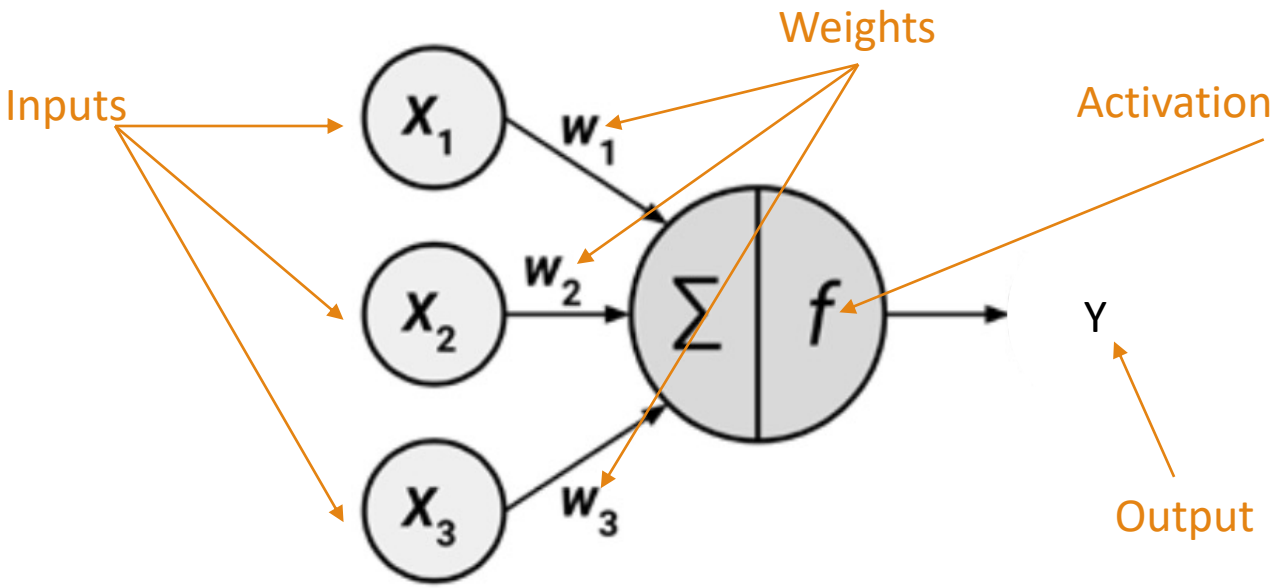
- Artificial Neural Network
  - At the core of deep learning are neural networks: mathematical entities capable of representing complicated functions through a composition of simpler functions.
  - The term *neural network* is suggestive of a link to the way our brain works.

Artificial Neural Network

- The basic building block of these complicated functions is the *neuron.*
  - A linear transformation of the input (for example, multiplying the input by a number [the *weight*] and adding a constant [the *bias*]) followed by the application of a fixed nonlinear function (referred to as the *activation function*).

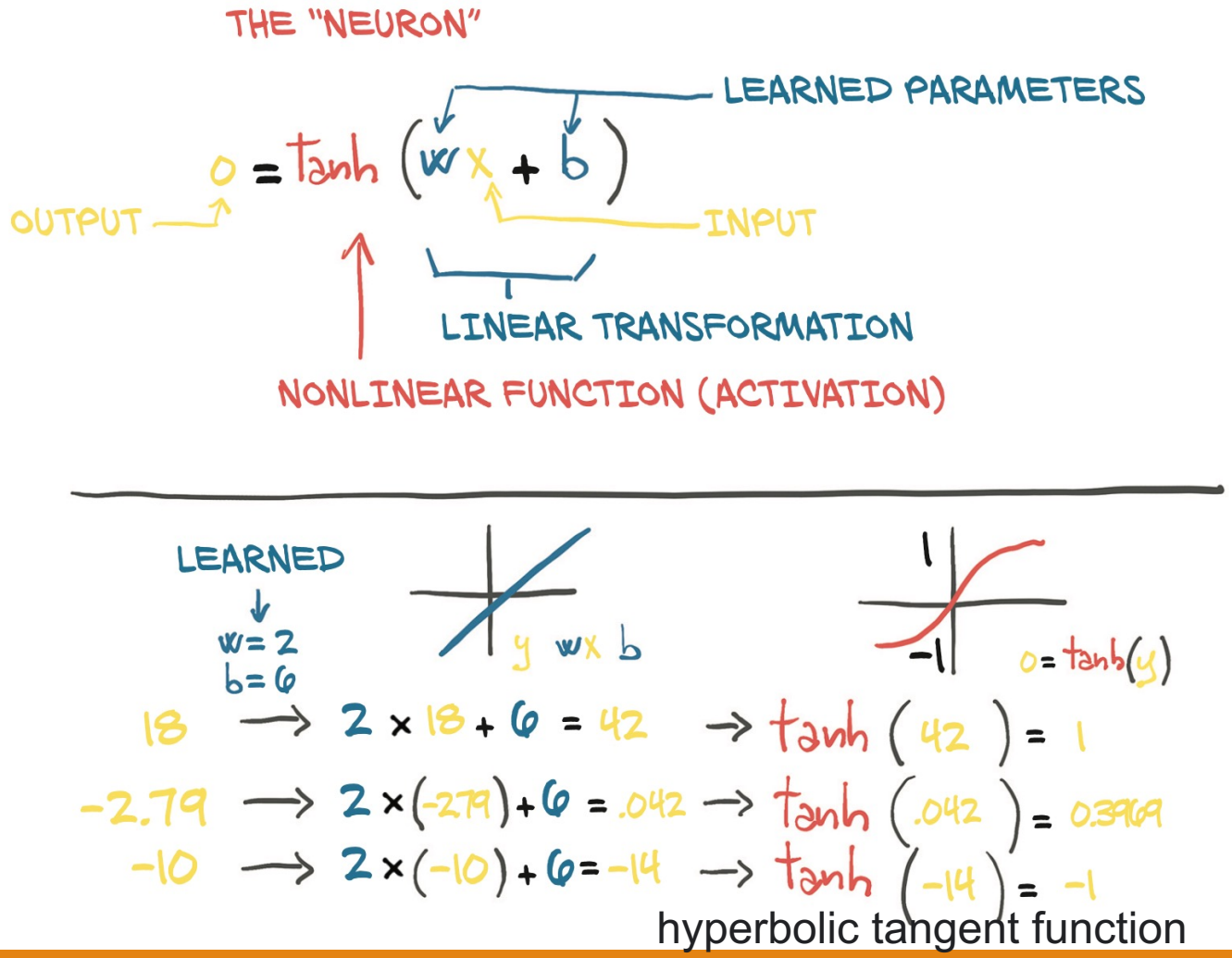
Artificial Neural Network

- The model of a single artificial neuron can be understood in terms very similar to the biological model.



$$y(x) = f\left(\sum_{i=1}^{n} w_i x_i\right)$$

Artificial Neural Network



hyperbolic tangent function

- Mathematically, we can write this out as

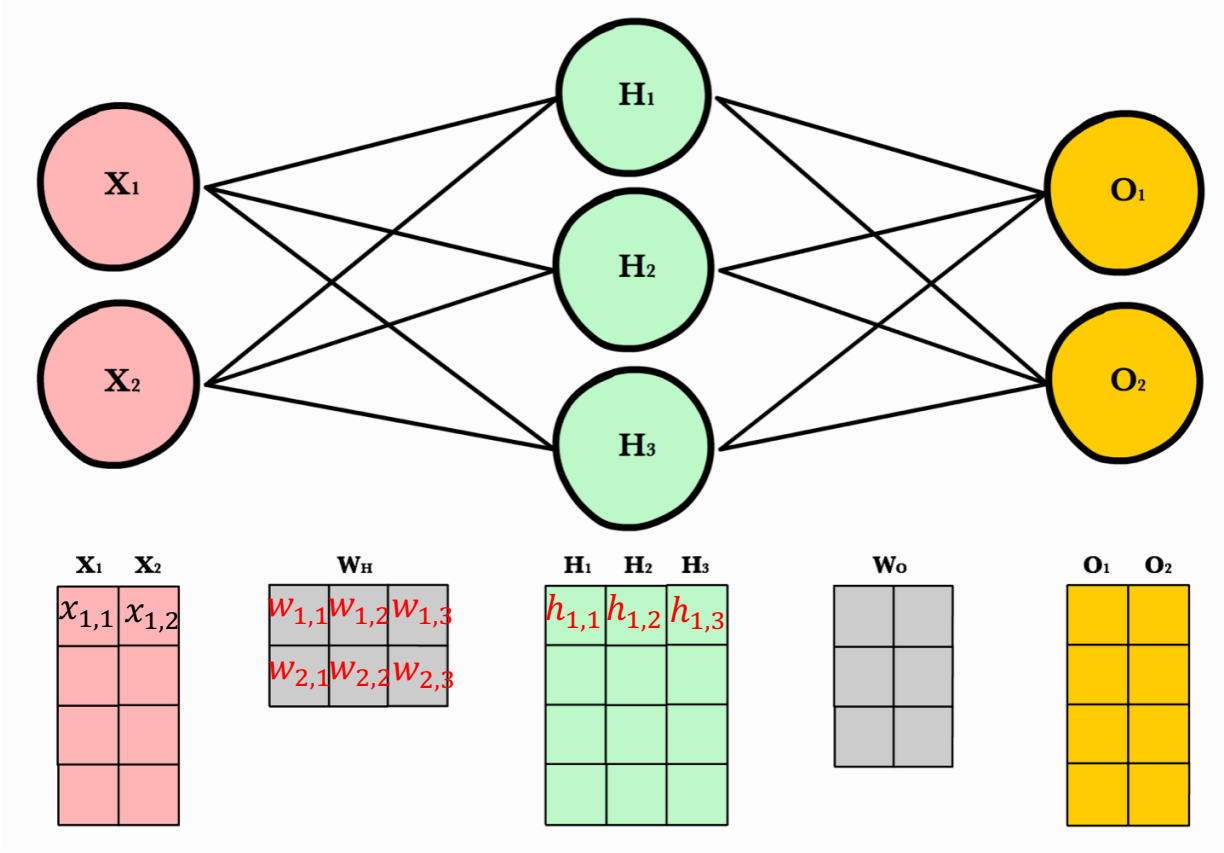
$$o = f(w * x + b),$$

with *x* as our input, *w* our weight or scaling factor, and *b* as our bias. *f* is our activation function.

- In general, *x* is a vector, and *w* is a matrix.

Artificial Neural Network

- A network with 2 input nodes, 3 hidden nodes, and 2 output nodes.



From https://ml-cheatsheet.readthedocs.io/en/latest/forwardpropagation.html

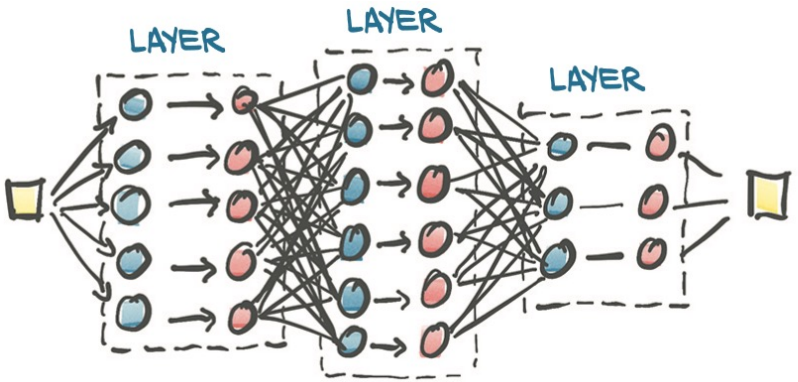
$f(\ x_{1,1} * w_{1,1} + x_{1,2} * w_{2,1} + b_1\ ) \rightarrow h_{1,1}$
$f(\ x_{1,1} * w_{1,2} + x_{1,2} * w_{2,2} + b_2\ ) \rightarrow h_{1,2}$
$f(\ x_{1,1} * w_{1,3} + x_{1,2} * w_{2,3} + b_3\ ) \rightarrow h_{1,3}$

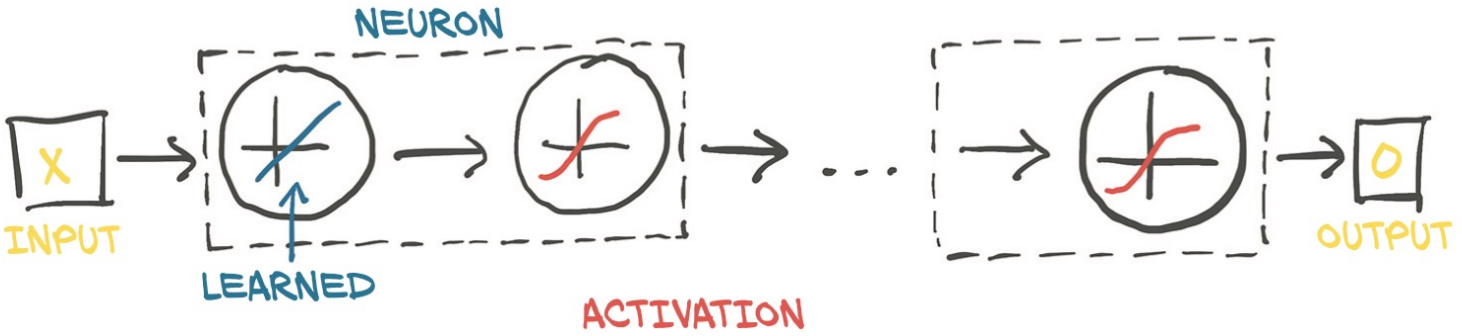
X (*nx2*) **W**(*2x3*) **H**(*nx3*)

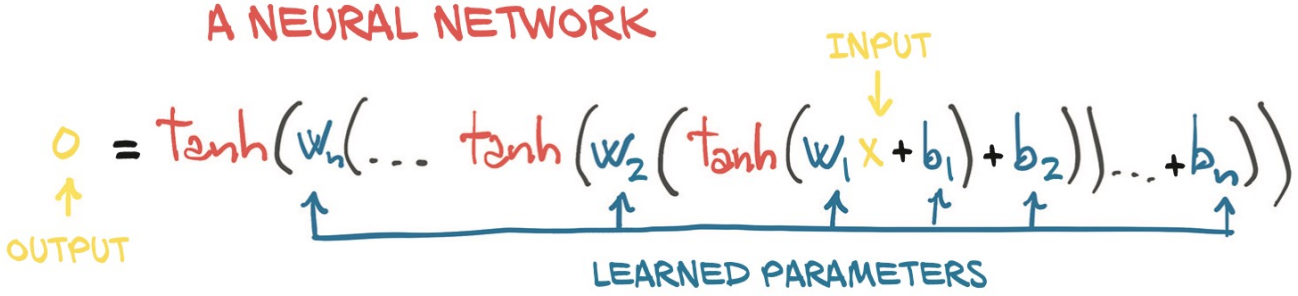
3-dimentional hidden space

Artificial Neural Network

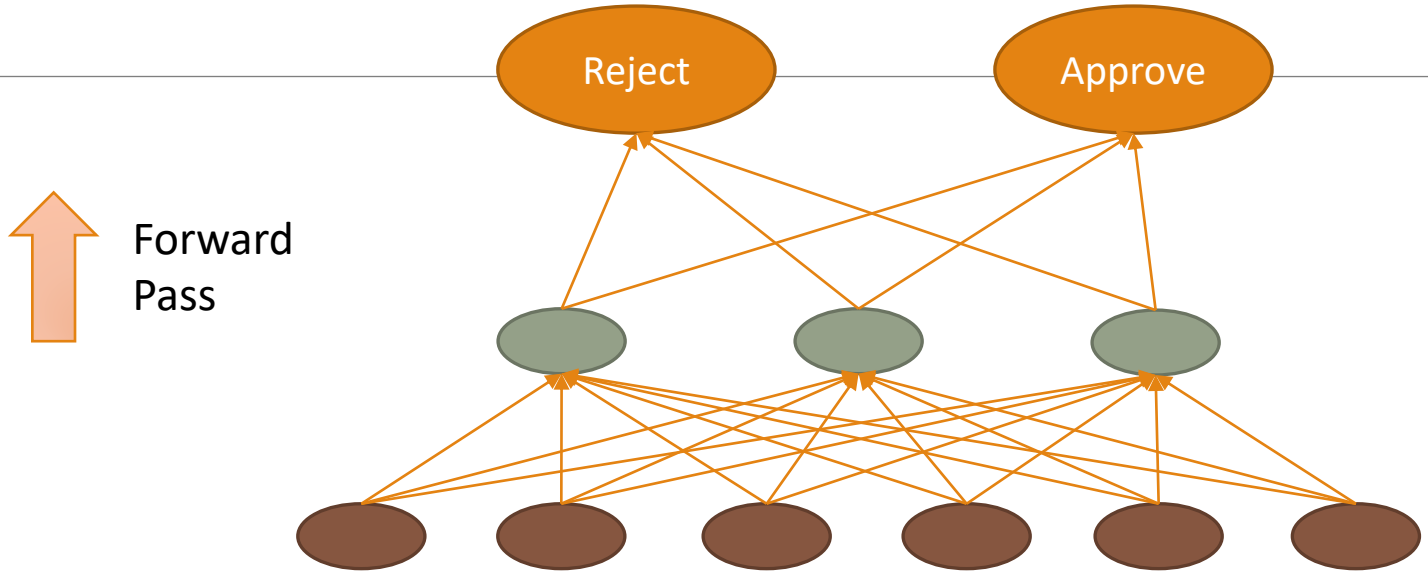
- Composing a multilayer network
  - Output of a layer of neurons is used as an input for the following layer







Neural Network: Forward Pass



Fully Connected Network

CID	Credit Score	Income (10,000)	Term	Age	Gender	Number of Children	Status
1	6	2	1	35	0	0	Reject
2	7.5	5	3	38	0	2	Approve
3	4	3.5	2	23	1	4	Reject
…							
100000	8	2.5	1	35	0	1	Approve

Artificial Neural Network

- The simplest unit in (deep) neural networks is a linear operation followed by an activation function.
- The **activation function** plays two important roles
  - It allows the output function to **have different slopes at different values**; neural networks can approximate arbitrary functions
  - At the last layer of the network, it has the role **of concentrating the outputs of the preceding linear operation into a given range** (e.g., Sigmoid and Softmax functions)

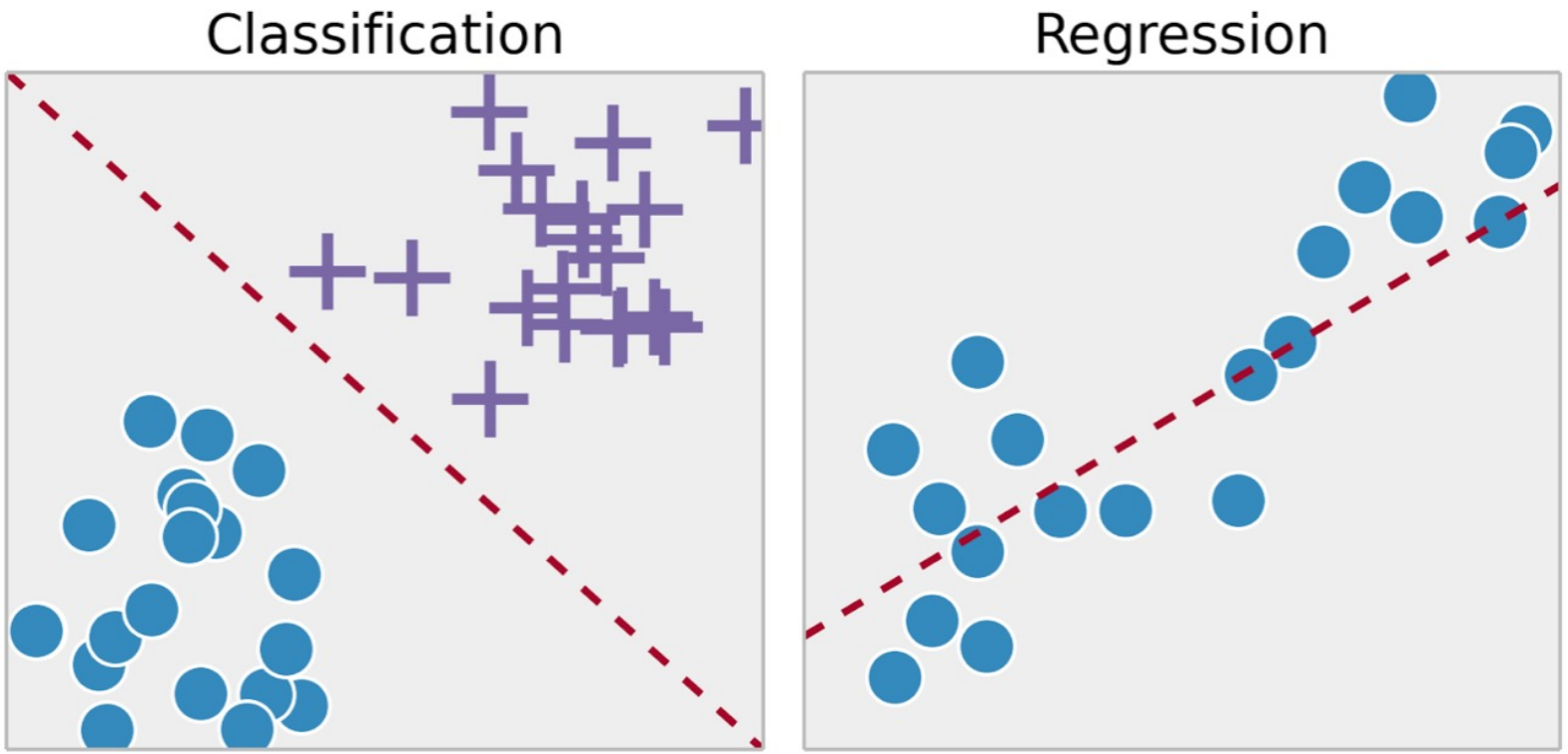
Artificial Neural Network

- In linear regression, the relationship between independent variable and the dependent variable is assumed to be **linear**. Yet this may not necessarily be true.
  - The effect of age on medical cost may not be constant throughout all the age values
  - Activation function introduce the nonlinearity into the model

```
Residuals:
        Min          1Q      Median          3Q         Max
-30302.4407 -1343.2315   985.6509   2988.446   11148.5211


Coefficients:
                      Estimate   Std. Error  t value   p value
_intercept       -11836.117831  1191.289921  -9.9355  0.000000
age                 256.402924    10.186903  25.1699  0.000000
bmi                 335.414456    16.372545  20.4864  0.000000
children            472.939931   166.843442   2.8346  0.004687
sex_male            -47.692891   400.980436  -0.1189  0.905348
smoker_yes        23435.046128   501.984750  46.6848  0.000000
region_northwest   -561.353117   545.419326  -1.0292  0.303645
region_southeast   -995.537125   551.901974  -1.8038  0.071580
region_southwest   -798.824371   541.710520  -1.4746  0.140648
---
R-squared:  0.73096,    Adjusted R-squared:  0.72864
F-statistic: 314.82 on 8 features
```

Artificial Neural Network



Artificial Neural Network

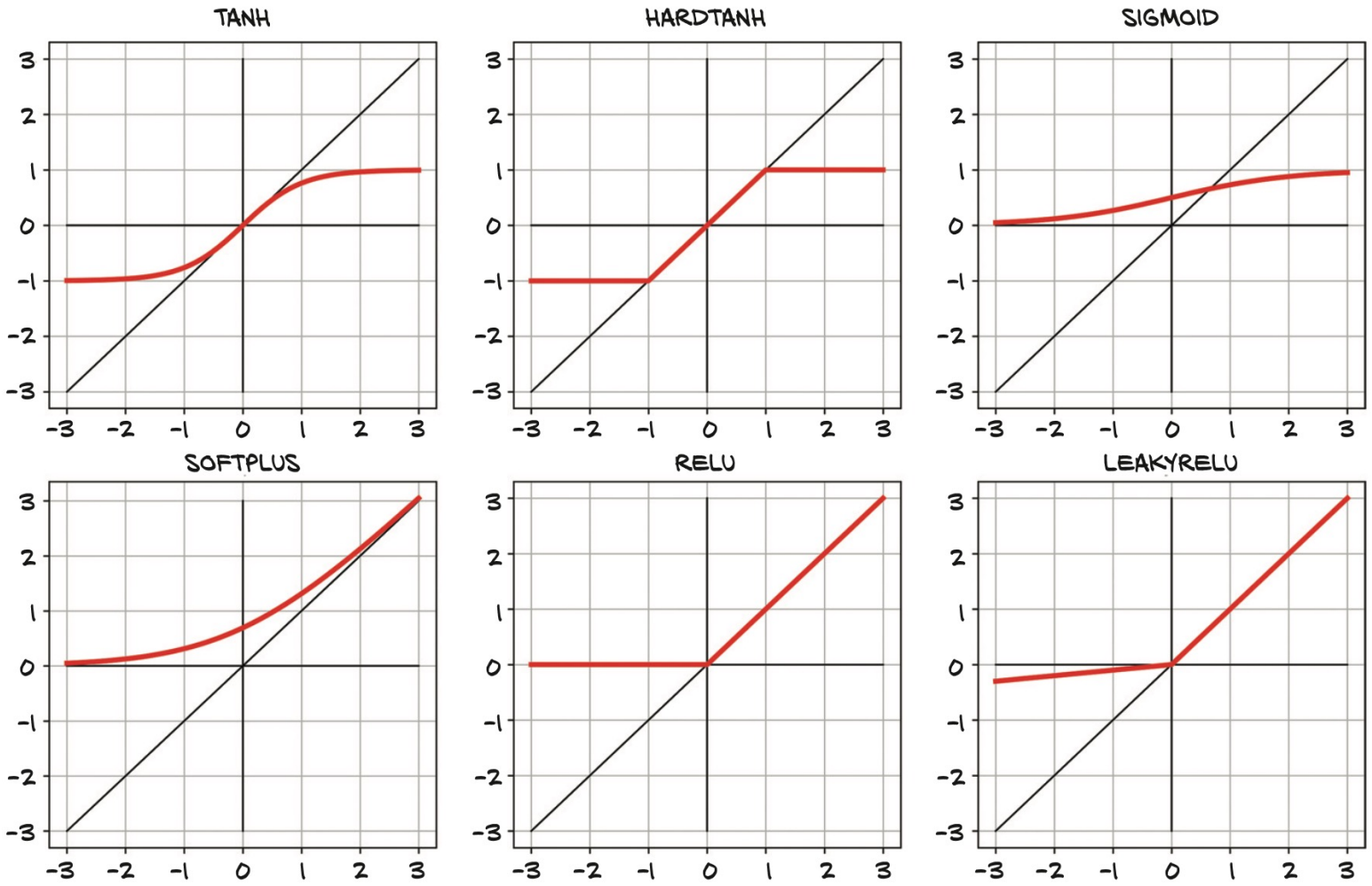
- Activation function concentrates the outputs of the preceding linear operation into a given range

  - Sigmoid function: $S(x) = \frac{1}{1 + e^{-x}}$

  - Softmax function: $\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$

Category	Output	e^{z_i}	Softmax
Reject	-1	0.368	0.047
Approve	2	7.39	0.953

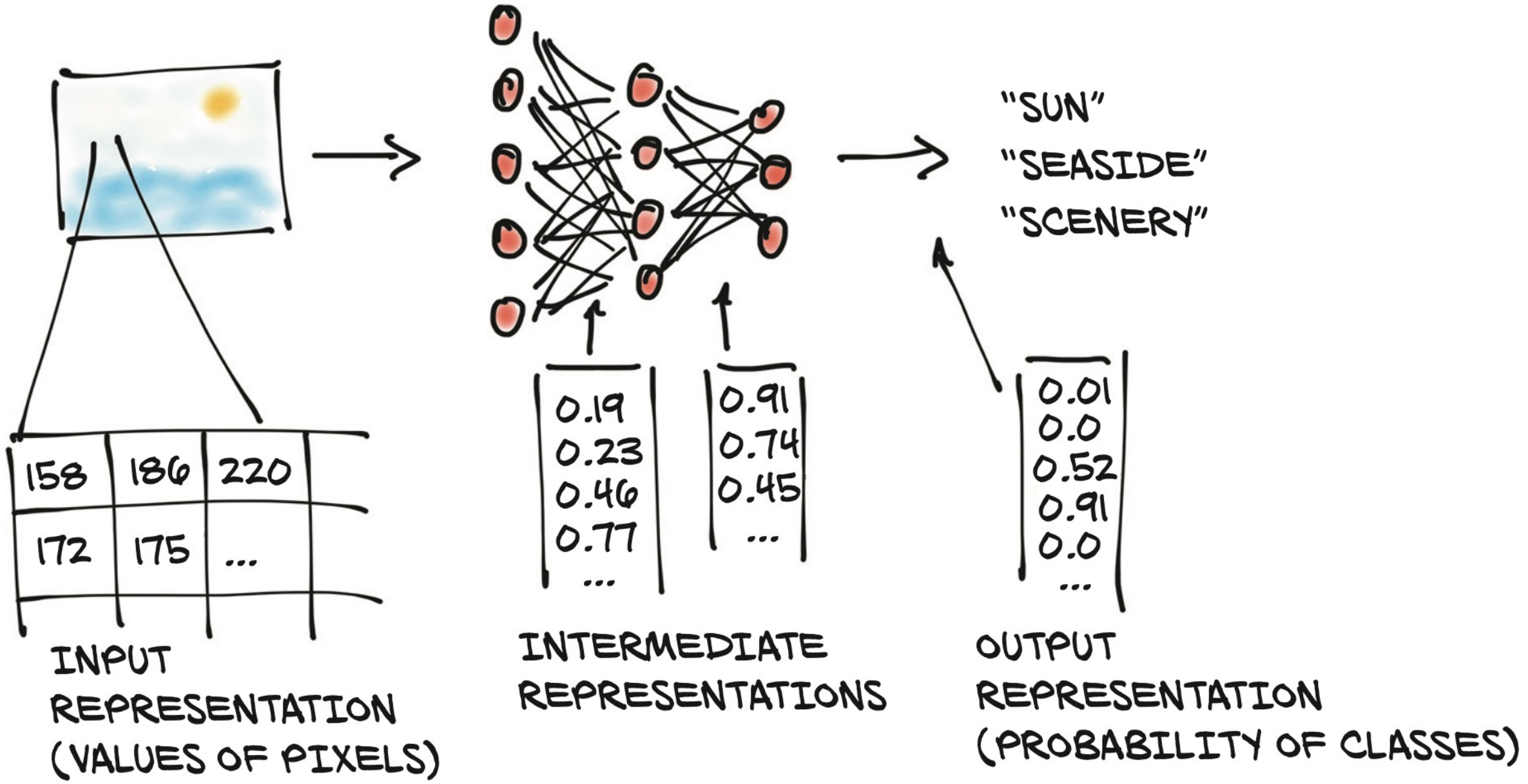
Artificial Neural Network



Fundamental Building Block: Tensors

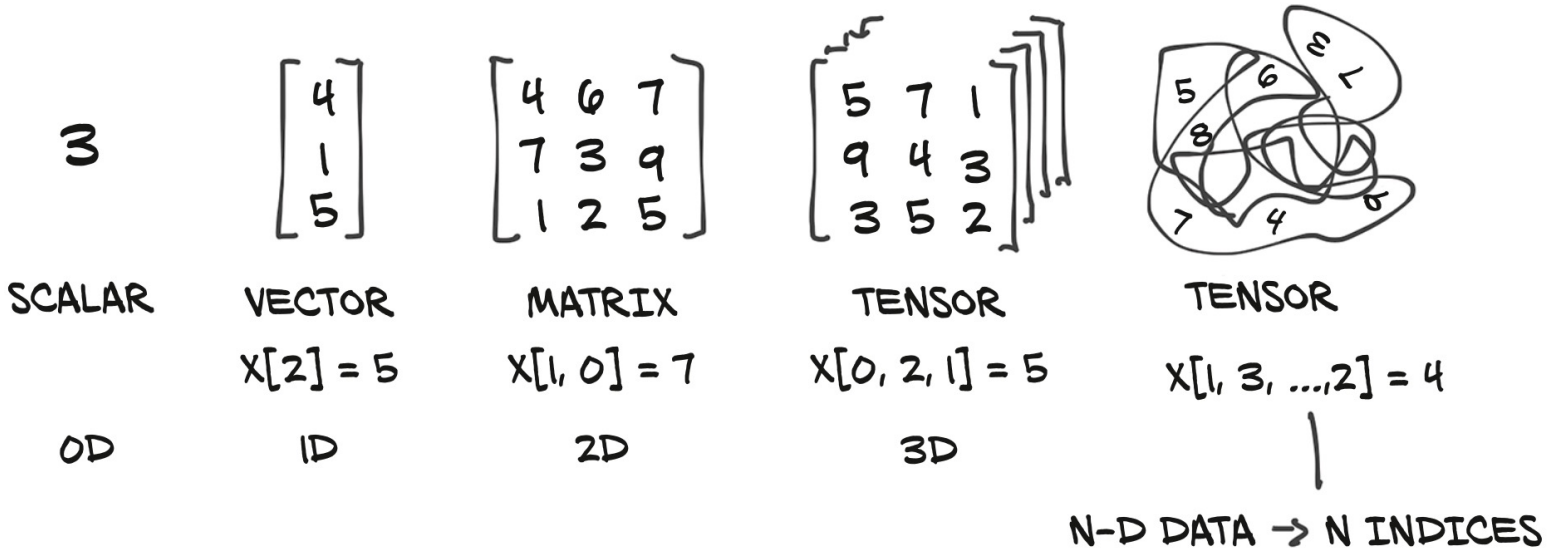
The World as Floating-Point Numbers

- Floating-point numbers and their manipulation are at the heart of modern AI.
  - Floating-point numbers are the way a network deals with information
  - Encode real-world data into something (***tensors***) digestible by a network and then decode the output back to something we can understand
  - **Neural network: a function describing how inputs are mapped to the outputs**


"SUN"
"SEASIDE"
"SCENERY"
158 186 220
172 175 ...
0.19
0.23
0.46
0.77
...
0.91
0.74
0.45
...
0.01
0.0
0.52
0.91
0.0
...
INPUT REPRESENTATION (VALUES OF PIXELS)
INTERMEDIATE REPRESENTATIONS
OUTPUT REPRESENTATION (PROBABILITY OF CLASSES)

Tensors: Multidimensional Arrays

- In the context of deep learning, **tensors** refer to the generalization of vectors and matrices to an arbitrary number of dimensions
  - Another name for the same concept is *multidimensional array*



Tensors: Multidimensional Arrays

- A **scalar** can represent something with one value like temperature or height: 20 degrees Celsius, 50 centimeters.
- A **vector (1D)** a tuple of one or more values called scalars.
  - A vector encodes a length and direction
- A **matrix (2D)** are two-dimensional arrays
  - It has rows and columns
- **Tensors** refer to the generalization of vectors and matrices to an arbitrary number of dimensions

Tensors: Multidimensional Arrays

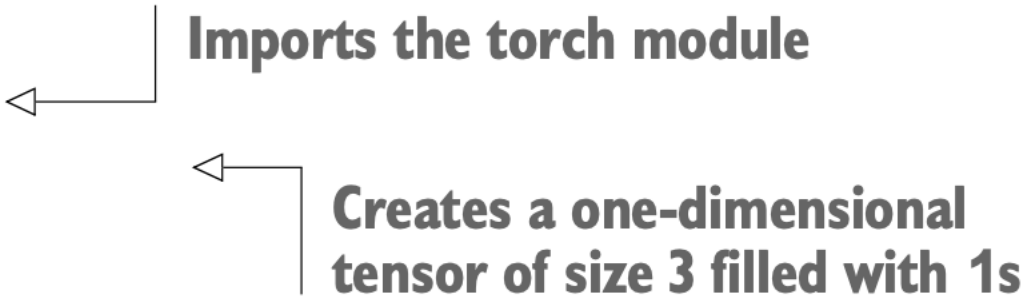
- A tensor is an array: that is, a data structure that stores a collection of numbers that are accessible individually using an **index**, and that can be indexed with multiple indices.
- Using the more efficient tensor data structure, different types of data can be represented.
  - Tabular data
  - Image data
  - Time series data
  - Text data

Create A Tensor

One-dimensional tensor

```
# In[4]:
import torch
a = torch.ones(3)
a

# Out[4]:
tensor([1., 1., 1.])
```



Create A Tensor

```
# Create a 2D tensor
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])
points
```

```
tensor([[4., 1.],
        [5., 3.],
        [2., 1.]])
```

```
# Access the first rwo of data
points[0] # The output is another tensor
```

```
tensor([4., 1.])
```

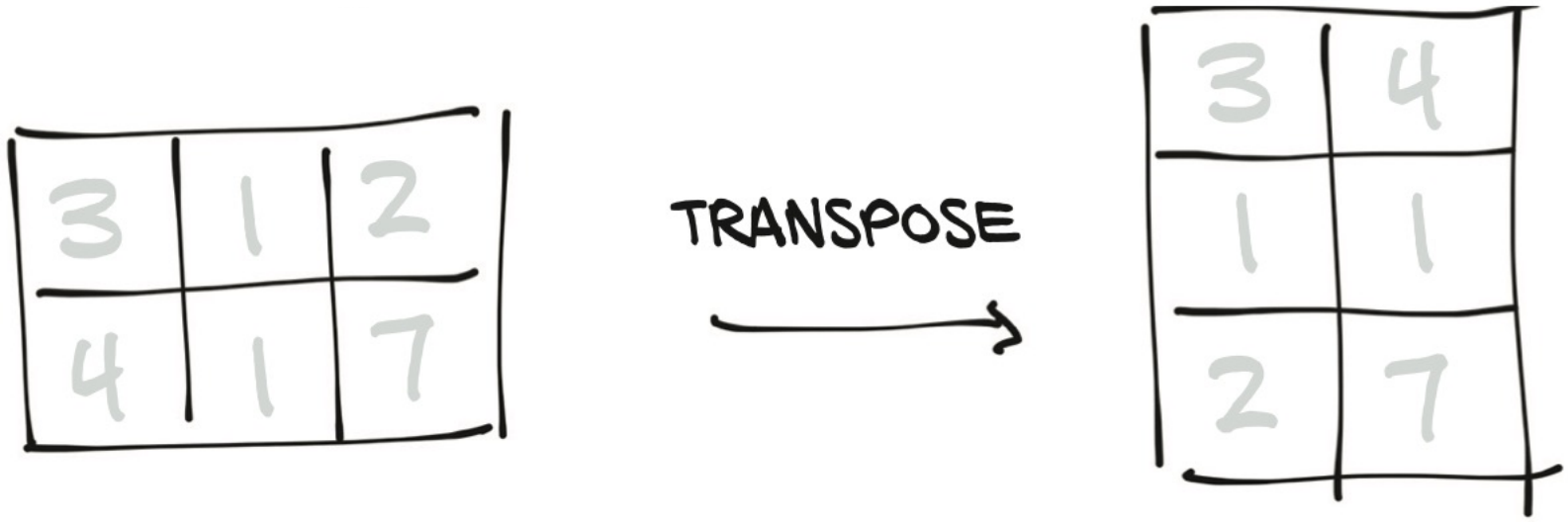
Tensor Element Types

- `torch.float32` or `torch.float`: 32-bit floating-point
- `torch.float64` or `torch.double`: 64-bit, double-precision floating-point
- `torch.float16` or `torch.half`: 16-bit, half-precision floating-point
- `torch.int8`: signed 8-bit integers
- `torch.uint8`: unsigned 8-bit integers
- `torch.int16` or `torch.short`: signed 16-bit integers
- `torch.int32` or `torch.int`: signed 32-bit integers
- `torch.int64` or `torch.long`: signed 64-bit integers
- `torch.bool`: Boolean

The default data type for tensors is 32-bit floating-point

Transpose A Tensor

- Transposing without copying



Transpose A Tensor

```
# Create a 2D tensor
points = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]])
points
```

```
tensor([[4., 1.],
        [5., 3.],
        [2., 1.]])
```

```
# Transpose the tensor
points_t = points.t()
points_t
```

```
tensor([[10.,  5.,  2.],
        [ 1.,  3.,  1.]])
```

```
# The transposed tensor shares the same storage
points_t[0,0] = 10.0
points
```

```
tensor([[10.,  1.],
        [ 5.,  3.],
        [ 2.,  1.]])
```

```
# Transpose in higher dimensions
some_t = torch.ones(3, 4, 5)
transpose_t = some_t.transpose(0, 2)
transpose_t.shape
```

```
torch.Size([5, 4, 3])
```

Transfer Tensor to GPU

- Every PyTorch tensor can be transferred to the GPU(s) in order to perform massively parallel, fast computations.
- In addition to dtype, a PyTorch Tensor also has the notion of device, which is where on the computer the tensor data is placed.
  - Create a tensor on the GPU by specifying the corresponding argument to the constructor:

```
points_gpu = torch.tensor([[4.0, 1.0], [5.0, 3.0], [2.0, 1.0]], device='cuda')
```

Transfer Tensor to GPU

- copy a tensor created on the CPU onto the GPU using the to method.

```
points_gpu = points.to(device='cuda')
```

- Doing so returns a new tensor that has the same numerical data, but stored in the RAM of the GPU, rather than in regular system RAM.
- We can also use the shorthand methods cpu and cuda instead of the to method

```
points_gpu = points.cuda()
points_cpu = points_gpu.cpu()
```

Save and Load Tensors

Save and load tensors

```
[ ]  #save the tensor
     torch.save(points, '/content/drive/MyDrive/DL_data/ourpoints.t')
```

```
[ ]  #load the tensor
     points = torch.load('/content/drive/MyDrive/DL_data/ourpoints.t')
```

Represent Tabular Data with Tensors

Tabular Data

- **Tabular data** refers to data that is organized in a table with rows and columns.

features

year	model	price	mileage	color	transmission
2011	SEL	21992	7413	Yellow	AUTO
2011	SEL	20995	10926	Gray	AUTO
2011	SEL	19995	7351	Silver	AUTO
2011	SEL	17809	11613	Gray	AUTO
2012	SE	17500	8367	White	MANUAL
2010	SEL	17495	25125	Silver	AUTO
2011	SEL	17000	27393	Blue	AUTO
2010	SEL	16995	21026	Silver	AUTO
2011	SES	16995	32655	Silver	AUTO

examples

If a variable represents a characteristic measured in numbers, it is unsurprisingly called **numeric variable**.

if a variable consists of a set of categories, the variable is called **categorical variable**.

Representing Tabular Data

Auction	Color	IsBadBuy	MMRCurrent	Size	TopThreeAm	VehBCost	VehicleAge	VehOdo	WarrantyCos	WheelType
ADESA	WHITE	No	2871	LARGE TRUC	FORD	5300	8	75419	869	Alloy
ADESA	GOLD	Yes	1840	VAN	FORD	3600	8	82944	2322	Alloy
ADESA	RED	No	8931	SMALL SUV	CHRYSLER	7500	4	57338	588	Alloy
ADESA	GOLD	No	8320	CROSSOVER	FORD	8500	5	55909	1169	Alloy
ADESA	GREY	No	11520	LARGE TRUC	FORD	10100	5	86702	853	Alloy
ADESA	SILVER	No	2659	COMPACT	GM	4100	7	73810	1455	Covers
ADESA	RED	No	4645	VAN	FORD	5600	5	85003	1633	Covers
ADESA	SILVER	No	4352	LARGE	GM	5900	5	88991	2152	Covers
ADESA	SILVER	No	5142	MEDIUM	GM	6600	5	80077	1373	Alloy
ADESA	MAROON	No	9983	MEDIUM	OTHER	7500	3	71952	1272	Alloy
ADESA	WHITE	No	4165	MEDIUM	OTHER	6200	4	23881	462	Covers

Representing Tabular Data (Categorical Variable)

- **Dummy coding**
  - Dummy coding: convert categorical variables into numeric format
    - Create **dummy variables** to replace the original categorical variable
    - Dummy coding for a binary variable: gender

$$\text{male} = \begin{cases} 1 & \text{if x} = \text{male} \\ 0 & \text{otherwise} \end{cases}$$

	Auction
Vehicle 1	OTHER
Vehicle 2	MANHEIM
Vehicle 3	ADESA

	Auction_MANHEIM	Auction_OTHER
Vehicle 1	0	1
Vehicle 2	1	0
Vehicle 3	0	0

Representing Tabular Data (Numeric Variable)

- **Data Transformation**
  - Standardization (Z-score standardization)

$$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$$

Numeric values will be rescaled to ensure the mean and the standard deviation to be 0 and 1, respectively

  - Normalization (max-min normalization)

$$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Numeric values will be rescaled to values between 0 and 1

Summary

- Artificial neural network
- Fundamental building block: tensors
- Represent tabular data with tensors